

Contemporary C++: *Learning Modern C++ in a Modern Way*



مدرس: سعید امراللہی بیوکی

Saeed Amrollahi Boyouki

www.Linkedin.com/in/saeed-amrollahi-boyouki

Agenda 16/30

Session 16 .Introduction to user-defined types: structures and enumerations

- Structures
- Designated initialization
- Enumeration and C++11 enumerations classes
- References and further readings
- Q&A

120 min (incl. Q & A)



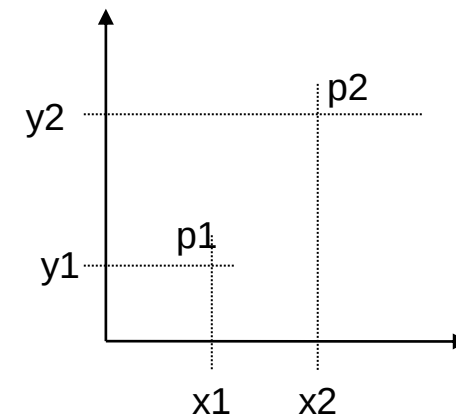
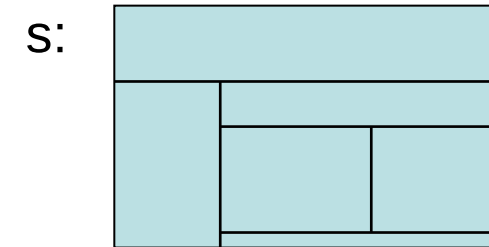
Structures

- An array is an aggregate of elements of the same type.
 - Homogeneous
- A **struct** is an aggregate of elements of arbitrary types.
 - Heterogeneous
- Data abstraction: Ignore irrelevant and unimportant things.

```
// represent point with two coordinates  
struct Point {  
    int x, y; // members  
};
```

- A **struct** is a scope.

```
struct Point {  
    int x, y; // local scope  
};  
int x; // another scope
```



Structures cont.

```
// type definition
struct Point {
    int x, y;
};
```

```
// object creation: memory allocation
struct Point { int x, y;} p1, p2; // two point objects
int a, b;
Point p3, p4; // two other points
```

- Struct initializer:

```
Point Center = {0, 0};
Point Max = { 1000, 1000 };
```

- Member selection operator or . (dot) operator.

structure-name . member

```
Point Center;
Center.x = 0;
Center.y = 0;
```

```
Point Max;
Max.x = Max.y = 1000;
```

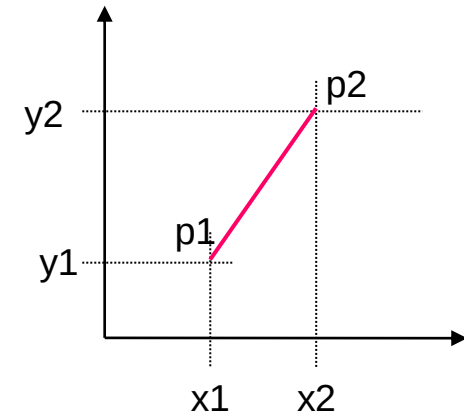
```
struct address {
    char* name; // "Ali Hasani"
    int number; // 61
    char* street; // "Valiasr"
    char* town; // "Tehran"
    long zip; // 79745
};
```

```
address ah;
ah.name = "Ali Hasani";
ah.number = 61;
// other members are uninitialized
```

Structures cont.

- Structures can be nested.

```
struct Line {  
    Point pt1;  
    Point pt2;  
};  
Line L;
```

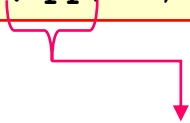


- Simple functions:

```
/* makepoint: make a point from x and y components */  
Point MakePoint(int x, int y)  
{  
    Point temp;  
    temp.x = x;  
    temp.y = y;  
  
    return temp;  
}  
  
const int XMAX = 1000;  
const int YMAX = 1000;  
Point p1 = MakePoint(0, 0);  
Point p2 = MakePoint(XMAX, YMAX);  
Point Middle = MakePoint((p1.x + p2.x) / 2, (p1.y + p2.y) / 2);
```

Pointer to structures

```
Point p = MakePoint(2, 3);  
Point* pp = &p;  
int xx = (*pp).x, yy = (*pp).y; // good
```



it is p

- Structure pointer dereference operator: ->

structure-name -> member

```
Point p = MakePoint(2, 3);  
Point* pp = &p;  
int xx = pp->x, yy = pp->y; // better
```

```
void print_name_and_family(address* p)  
{  
    // just print name and family  
    cout << p->name << '\n' << p->family << '\n';  
}
```

- Another example:

```
struct student_info {  
    string name;  
    double midterm, final;  
    vector<double> homework;  
};  
student_info myInfo;
```

Enumeration

- An **enumeration** is a type that can hold a set of values specified by the user.

```
enum Tag_nameop { enumerator1 = initop, enumerator2 = initop, ..., enumeratorn = initop };
```

```
enum { dark, light };  
enum Season { WINTER, SPRING, SUMMER, FALL };  
enum Color { RED, GREEN, BLUE };  
enum Bool { False, True }; // redundant
```

- The elements of an enumeration called **enumerator**. They are symbolic integral constants. They are r-value. Unlike constants, they are not addressable.
- By default, enumerator values are assigned increasing from 0.

```
enum { dark, light }; // dark == 0, light = 1  
enum Season { WINTER = 1, SPRING, SUMMER, FALL }; // SPRING == 2, ...  
enum Color { RED = 0, GREEN = 2, BLUE = 4 };
```

- Each enumeration is distinct type.

```
Season mySeason = SUMMER;  
Color c = YELLOW; // error  
Color c = SUMMER; // error  
Bool b = false; // error: can't convert from bool to Bool
```

- Enumerations are converted to integers for arithmetic operations.

Enumeration classes

- Conventional enumerations

```
enum Season { WINTER, SPRING, SUMMER, FALL };
enum TrafficLight { RED, YELLOW, GREEN };
enum Color { Red, Green, Blue };
Color c = RED; //error: two different enumerations
Season s = 3; // I mean FALL! error: no conversion from int to enum
int color_as_int = Blue; // implicit conversion
```

- Conventional enumerations have three problems:
 1. implicit conversion to int
 2. conventional enums export their enumerators to the surrounding scope, causing name clashes.
 3. the underlying type of an enum cannot be specified.
- enumeration classes are *strongly-typed* and *scoped*.

```
enum class TrafficLight { Red, Yellow, Green };
enum class Color { Red, Green, Blue };
TrafficLight CrossStreet = TrafficLight::Red;
Color RGB = Color::Red;
int c = RGB; // error
```

Enumeration classes continued

- Underlying type
- conventional enums and enumeration classes: compatibility

```
enum class TrafficLight { RED, YELLOW, GREEN }; // the underlying type is int, by default
enum class Color : char { red, blue }; // compact representation
enum class E : long long { E1= 1LL, E2 = 2LL, Ebig = 0xFFFFFFFFFFFFFFFF };
```

- Very important feature in embedded system programming

Rferences and further readings

cppreference.com. Aggregate initialization.

https://en.cppreference.com/w/cpp/language/aggregate_initialization.html. Retrieved September 2025

[Str00] Bjarne Stroustrup. The C++ Programming Language (Special Edition). Addison Wesley. Reading Mass. USA. February 2000.

[Str13] Bjarne Stroustrup. The C++ Programming Language. 4th edition. Addison Wesley. Reading Mass. USA. May 2013.

[Str22] Bjarne Stroustrup. A Tour of C++, 3rd edition, Addison-Wesley, September 2022.

Thanks for your patience ...

A man who asks a question is a fool for minute,
The man who does not ask, is a fool for a life.
- Confucius

Learning to ask the right (often hard) questions is an essential part of learning to think as a programmer.

- Bjarne Stroustrup *programming Principles and Practice Using C++, page 4.*

There is no stupid question, but there is stupid answer.
- Howard Hinnant

